

Reporte de Auditoría y Refactorización

Backend v1.0 · Julio 2026

RESUMEN EJECUTIVO

El proyecto se encontraba en buen estado general con arquitectura clara y separación de responsabilidades consistente. Los 93 tests existentes cubrían los flujos principales. Se realizaron 5 fases de auditoría: mapeo de dependencias, limpieza de código muerto, optimización, documentación y detección de bugs. Se identificaron y corrigieron 2 bugs (1 crítico, 1 medio), se eliminaron 5 imports muertos, se consolidó lógica duplicada y se aplicaron mejoras de rendimiento. El endpoint GET /api/admin/clientes/:id nunca funcionó desde su creación — queda corregido. Los 93 tests siguen pasando tras todos los cambios.

22	7	53
Archivos revisados	Archivos modificados	Líneas eliminadas
2	93/93	5
Bugs corregidos	Tests	Commits

1. DESCRIPCIÓN DEL PROYECTO

Stack tecnológico

Capa	Tecnología	Versión / Detalle
Runtime	Node.js / Express	4.x
Base de datos	PostgreSQL	14+ · 38 tablas · 45 funciones almacenadas
Autenticación	JWT + sesión anónima	Bearer token + x-session-key header
Pagos	PayPal / Stripe / SPEI	Orders v2 · PaymentIntents · token compartido
Email	Nodemailer	SMTP — fire-and-forget
PDF export	PDFKit	Reportes admin (clientes / productos)
Frontend	HTML estático	Servido por Express, sin bundler
Tests	Jest + Supertest	93/93 pasando · --runInBand

Estructura de archivos auditados

Archivo	Responsabilidad
src/app.js	Entry point, middlewares, rutas, error handler
src/config/db.js	Pool pg — helper query()
src/middlewares/auth.js	verificarToken, soloAdmin, tokenOpcional
src/middlewares/upload.js	Multer — imágenes, avatares, logo, CSV
src/routes/auth.routes.js	Registro, login, perfil, recuperación de contraseña
src/routes/productos.routes.js	Catálogo, categorías, marcas, reseñas
src/routes/carrito.routes.js	Carrito anónimo y autenticado
src/routes/pedidos.routes.js	Checkout, detalle, factura, cancelación, soporte
src/routes/usuarios.routes.js	Foto, direcciones, favoritos, notificaciones
src/routes/admin.routes.js	Panel admin completo (requiere soloAdmin)
src/routes/pagos.routes.js	SPEI, PayPal, Stripe
src/routes/contacto.routes.js	Formulario de contacto
src/controllers/auth.controller.js	Lógica de autenticación
src/controllers/productos.controller.js	Catálogo y reseñas
src/controllers/carrito.controller.js	Operaciones de carrito
src/controllers/pedidos.controller.js	Creación de pedidos y emails
src/controllers/admin.controller.js	Panel admin — 35 handlers
src/controllers/pagos.controller.js	Webhooks y operaciones de pago
src/services/email.service.js	4 plantillas de email transaccional
src/services/factura.service.js	Generador de factura HTML imprimible
src/services/paypal.service.js	PayPal Orders v2 — OAuth2 + captura
src/services/stripe.service.js	Stripe PaymentIntents + webhook verify

2. MAPA DE DEPENDENCIAS

Todas las dependencias son directas (require explícito). No se detectaron dependencias circulares.

Módulo (exporta)	Exports principales	Consumido por
config/db.js	pool, query	Todos los controladores y algunas rutas
middlewares/auth.js	verificarToken, soloAdmin, tokenOpcional	Todas las rutas
middlewares/upload.js	uploadImage, uploadAvatar, uploadCsv, uploadPdf	admin.routes, usuarios.routes
services/email.service	enviarCorreo*, enviarNotif*, credenciales...	auth.routes, pedidos.controller, admin.controller
services/factura.service	generarFacturaHTML	pedidos.routes
services/paypal.service	credenciales*, crearOrden, capturar*, getCfg	pagos.controller
services/stripe.service	crearPaymentIntent, verificarWebhook, getPublicKey	pagos.controller
controllers/auth	registro, login, perfil, actualizarPerfil, cambiarPassword	auth.routes
controllers/productos	listar, detalle, categorias, marcas, resenas, crearResena	productos.routes
controllers/carrito	obtener, agregar, eliminar, actualizar, fusionar	carrito.routes
controllers/pedidos	crear, listar, detalle, metodos_envio, metodos_pago	pedidos.routes
controllers/admin	35 handlers exportados	admin.routes
controllers/pagos	estadoPago, SPEI/PayPal/Stripe handlers	pagos.routes

Dependencias externas (npm)

Paquete	Uso
express	Framework HTTP
pg	Cliente PostgreSQL (pool de conexiones)
bcryptjs	Hash de contraseñas
jsonwebtoken	Firma y verificación de JWT
nodemailer	Envío de emails SMTP
multer	Upload de archivos (disk + memory storage)
pdfkit	Generación de PDFs (reportes admin)
stripe	SDK oficial de Stripe
helmet	Headers de seguridad HTTP
cors	Política CORS
morgan	Logger de requests HTTP
express-rate-limit	Rate limiting en auth
dotenv	Variables de entorno

3. HALLAZGOS DE LA AUDITORÍA

FASE 2 — Código muerto confirmado

ID	Archivo	Hallazgo	Acción
D1	factura.service.js	Imports nunca usados: query, exec (child_process), path, fs	Eliminados
D2	factura.service.js	Comentario decía "convierte a PDF usando Puppeteer" — código solo genera HTML	Corregido

FASE 3 — Optimizaciones aplicadas

ID	Archivo	Hallazgo	Acción
O1	auth.routes.js	require("bcryptjs") dentro de handler — re-evaluado en cada request	Movido al top
O2	admin.controller.js	require("bcryptjs") y require("pdfkit") dentro de handlers	Movidos al top
O3	carrito.routes.js	require("../config/db") inline dentro de route handler GET /id	Movido al top
O4	admin.controller.js	Slug generado con la misma regex en 3 lugares (guardarProducto, importar crearCategoría)	generarSlug()
O5	usuarios.routes.js	es_predeterminada ? true : false — doble negación redundante × 2	!es_pred...
O6	admin.controller.js	guardarConfigNotif: N queries secuenciales en loop for...await	Promise.all

FASE 5 — Bugs detectados

ID	Severidad	Descripción	Estado
B1	CRÍTICO	fn_detalle_cliente no existe en BD — GET /api/admin/clientes/:id siempre devuelve 500	' Corregido
B2	MEDIO	marcarPrincipallimagen: dos UPDATES sin transacción — si el 2º falla, ninguna imagen queda como principal	' Corregido
B3	BAJO	guardarConfigNotif: loop secuencial interrumpe claves posteriores si una falla	' Mejorado
B4	BAJO	Avatares anteriores no se eliminan del disco al subir uno nuevo	Pendiente
B5	ALTO	stripe_webhook_secret vacío — webhooks Stripe siempre rechazan con 400	Config ext.
B6	ALTO	paypal_webhook_id vacío — webhook PayPal pasa sin verificar firma	Config ext.

4. CAMBIOS APLICADOS POR LOTE

Lote 1 — Limpieza (riesgo bajo)

Commit: 7f63cb7 · 6 archivos · " 26 líneas / +18 líneas

Archivo	Cambio
factura.service.js	Eliminados 4 imports muertos (query, exec, path, fs) y comentario engañoso sobre Puppeteer
auth.routes.js	require("bcryptjs") movido al top del módulo
carrito.routes.js	require("../config/db") inline movido al top del módulo
auth.controller.js	Normalización de imports (cosmético)
admin.controller.js	require("bcryptjs") y require("pdfkit") movidos al top del módulo
usuarios.routes.js	es_predeterminada ? true : false !' !!es_predeterminada (×2)

Lote 2 — Optimización (riesgo bajo)

Commit: 817be87 · 1 archivo · " 17 líneas / +22 líneas

Ubicación	Cambio
admin.controller.js (guardarProducto)	Slug de 3 líneas reemplazado por generarSlug(nombre)
admin.controller.js (importarProductos)	Idem — la versión unificada también colapsa guiones múltiples
admin.controller.js (crearCategoria)	Idem
admin.controller.js (guardarConfigNotif)	Loop for...await !' Promise.all — N keys en paralelo

Lote 3 — Bug medio (riesgo bajo-medio)

Commit: 0f977fb · 1 archivo · " 6 líneas / +18 líneas

- Se importa pool desde config/db (ya estaba exportado).
- marcarPrincipallimagen: ambos UPDATEs dentro de BEGIN / COMMIT / ROLLBACK.
- Bloque finally garantiza client.release() incluso si ROLLBACK falla.
- Escenario corregido: fallo en segundo UPDATE ya no deja el producto sin imagen principal.

Lote 4 — Bug crítico (riesgo medio)

Commit: 124f949 · 1 archivo · " 4 líneas / +24 líneas

- fn_detalle_cliente nunca existió en PostgreSQL — el endpoint era completamente inoperable.
- Reemplazado por SQL inline: JOIN usuarios !' pedidos, agrega total_pedidos y total_gastado.
- Consistente con el patrón de listarClientes ya existente en el mismo controlador.
- Se agrega console.error al catch para visibilidad futura.

Campos que ahora devuelve GET /api/admin/clientes/:id:

id · nombre · apellidos · email · telefono · rfc · razon_social · tipo · activo · bloqueado · motivo_bloqueo · avatar_url · notas_internas · ultimo_login · created_at · total_pedidos · total_gastado + pedidos_recientes (últimos 10)

5. ESTADÍSTICAS FINALES

Métrica	Valor
Archivos fuente revisados	22
Archivos modificados	7
Líneas eliminadas	53
Líneas agregadas (fixes + refactor)	82
Imports muertos eliminados	5
Requires dinámicos movidos al top	4
Duplicaciones eliminadas	1 (generarSlug en 3 lugares !' función única)
Bugs críticos corregidos	1 (fn_detalle_cliente inexistente)
Bugs medios corregidos	1 (race condition en imágenes)
Commits generados	5
Tests antes / después	93/93 !' 93/93 '
Estado del repositorio	Limpio — origin/master al día

6. DEUDA TÉCNICA DETECTADA (NO MODIFICADA)

Los siguientes elementos fueron identificados pero no modificados por requerir decisión del equipo o configuración externa.

Área	Descripción	Impacto
Tablas huérfanas	blog_articulos, cotizaciones, newsletter, tarjetas_guardadas — infraestructura de BD completa sin rutas ni controladores	Bajo
fn_admin_listar_clientes	Función almacenada que existe en BD pero nunca se llama desde Node.js	Bajo
Avatar sin limpieza	POST /api/usuarios/foto guarda el nuevo avatar pero no elimina el anterior — acumulación en disco	Medio
CSP desactivado	helmet({ contentSecurityPolicy: false }) — conveniente pero reduce protección en producción	Medio
Stripe webhook secret	stripe_webhook_secret_test/live vacíos — todos los webhooks de Stripe rechazan con 400	Alto
PayPal webhook ID	paypal_webhook_id_test/live vacíos — webhook PayPal procesa sin verificar firma	Alto
43 filas sin seccion	Claves de versión anterior de configuracion con seccion = NULL — no causan error pero ensucian la tabla	Bajo

7. OPORTUNIDADES FUTURAS

#	Oportunidad	Descripción	Prioridad
1	Activar webhooks de pago	Configurar stripe_webhook_secret_test y paypal_webhook_id_test desde sus respectivos dashboards. Sin estos valores los pagos solo se confirman por el	Alta
2	Limpieza de avatares	Al subir nueva foto de perfil, eliminar el archivo anterior con fs.unlink para evitar acumulación indefinida en /public/uploads/avatares/	Media
3	CSP progresivo	Definir Content-Security-Policy que permita dominios de PayPal (paypal.com, paypalobjects.com) y Stripe (js.stripe.com). Aumenta significativamente la	Media
4	Limpiar configuracion	Las 43 filas sin seccion son basura de una versión anterior. Crear una migración que las archive en una tabla separada o las elimine tras verificar	Baja
5	Implementar cotizaciones	Las tablas cotizaciones, cotizacion_items y las funciones fn_crear_cotizacion, fn_responder_cotizacion existen y están funcionales. Solo falta la capa	Media
6	Rate limiting en webhooks	Los endpoints /api/pagos/*/webhook no tienen rate limiting. Aunque están protegidos por token/firma, limitar a IPs de PayPal/Stripe/STP añade una capa adicional de defensa.	Media

8. CONCLUSIONES

El backend de MBS Comunicaciones presenta una arquitectura sólida y coherente. La separación entre rutas, controladores y servicios es clara. La delegación de lógica de negocio a funciones almacenadas de PostgreSQL es consistente y facilita el mantenimiento. El conjunto de tests proporciona una red de seguridad confiable para futuros cambios.

Los cambios aplicados en esta auditoría eliminaron código muerto confirmado, consolidaron lógica duplicada, mejoraron la inicialización de módulos y corrigieron dos bugs reales: uno crítico (endpoint completamente inoperable) y uno medio (operación no atómica que podía dejar datos en estado inconsistente). En ningún caso se modificó el comportamiento funcional del sistema ni los contratos de la API.

Las dos pendientes de alta prioridad (webhook secrets de Stripe y PayPal) son de configuración externa, no de código. Una vez configurados desde los dashboards de cada proveedor, el sistema estará completamente operativo para confirmar pagos tanto de forma síncrona como asíncrona.

